CMPT 354 Mini Project
Library Database

Steps 1-3: Project Specifications, E/R Diagrams, BCNF Analysis

Person A: Item / Loan / Fine / AcquisitionCandidate

Person B: Person / Room / Event / AudienceGroup

Step 1: Project Specifications

The library holds print books, online books, magazines, scientific journals, and recordings; people can borrow and return items and may be fined for late returns; the library also holds events (book clubs, art shows, film screenings, and others) recommended for specific audiences and held in library rooms, which people can attend for free; the library keeps personnel records and tracks candidate items for future acquisition. The rest of the domain is specified below, split by the two halves of the schema.

Items

The library holds several kinds of items available to patrons: print books, online (e-) books, magazines, journals, and recordings (e.g., CDs, vinyl records, DVDs). Every item, regardless of type, has a title, the method by which it was acquired (purchased or donated), the date it was added to the collection, and a current status (available, checked out, or lost). Print books and online books share details such as ISBN and author, but differ in that a print book has a physical shelf location while an online book has a file format and an access URL instead. Magazines and journals are periodicals: magazines are identified by issue number, publication date, and publisher, while journals are identified by volume, issue number, and subject area. Recordings have an artist, a format, and a duration.

Borrowing, Returns, and Fines

Members can borrow items from the library. Each time a member borrows an item, this creates a loan with a due date; when the item comes back, a return date is recorded. The same member can borrow the same item more than once over time, so each borrowing is tracked as its own event rather than a single fixed relationship between a member and an item. If a loan is not returned by its due date, it becomes overdue and the library may issue a fine for that specific loan, with an amount owed, the date it was issued, and (once settled) the date it was paid. Not every loan results in a fine; only those returned late or never returned.

Future Acquisitions

The library also keeps a record of items being considered for future acquisition. Each proposed item has a title, a format, an estimated cost, the date it was proposed, and a status indicating whether the proposal is still pending, has been approved, or was rejected. A proposal may optionally be linked to the person who suggested it, if that information is known. Once approved, an acquisition candidate is expected to eventually become an actual item in the collection, but that transition is handled by library staff rather than being an automatic part of the data itself.

People

The library keeps a record of every person it interacts with: each person has a name, a contact email, a phone number, and an address. Email addresses are required and no two people may share one, since email is how the library reaches a person about loans, fines, and event registrations. A person may play any combination of three roles at the library, or none at all. A

member is a person who has signed up to borrow items; the library records the date the membership started and its current status (active, expired, or suspended). A staff member is a person employed by the library, with a job role, a hire date, and a salary that cannot be negative; every staff member except the head of the library reports to exactly one supervisor, who must themselves be a staff member, and nobody can be recorded as their own supervisor, nor can the reporting chain loop back on itself, since supervision must ultimately lead up to the head of the library. A volunteer is a person who donates their time, with the date they started and a running total of hours logged, which likewise cannot be negative. These roles overlap freely: the same person can be a member and a staff member, or a member and a volunteer, at the same time, and a person can also hold no role at all, for example someone who only donates an item or attends a public event without ever joining.

Rooms and Events

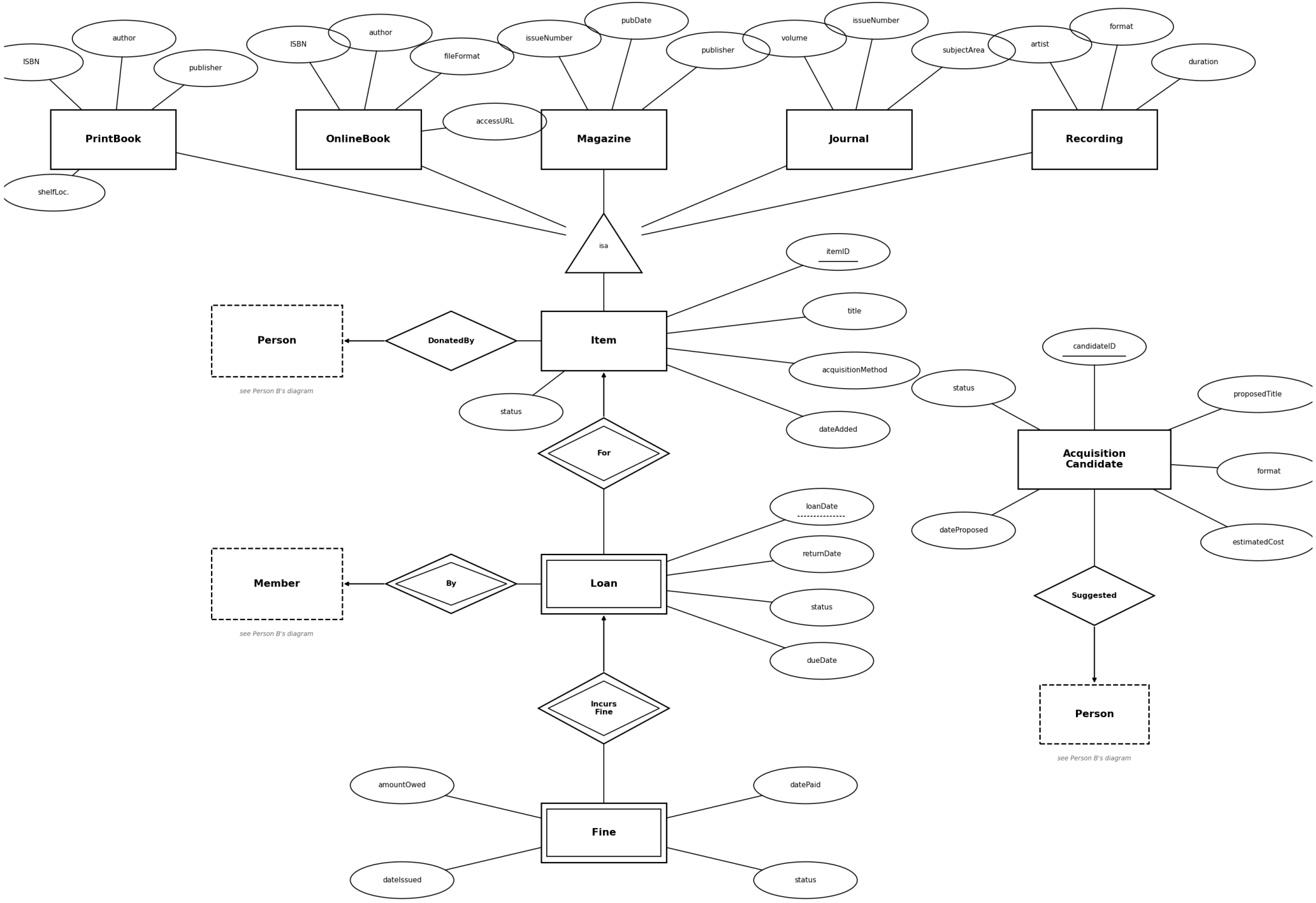
The library building contains a number of rooms, each with a name and a seating capacity that must be at least one. The library runs events such as book clubs, art shows, and film screenings (with an "other" category for anything else); each event has a title, an optional longer description, a type, a date, and a start and end time, where the end time must come after the start time. Every event is held in exactly one room: an event cannot exist without a room assignment. A room cannot be double-booked: two events in the same room on the same day must not overlap in time, so a given room hosts at most one event at any moment (and in particular, no two events in one room can share the same date and start time).

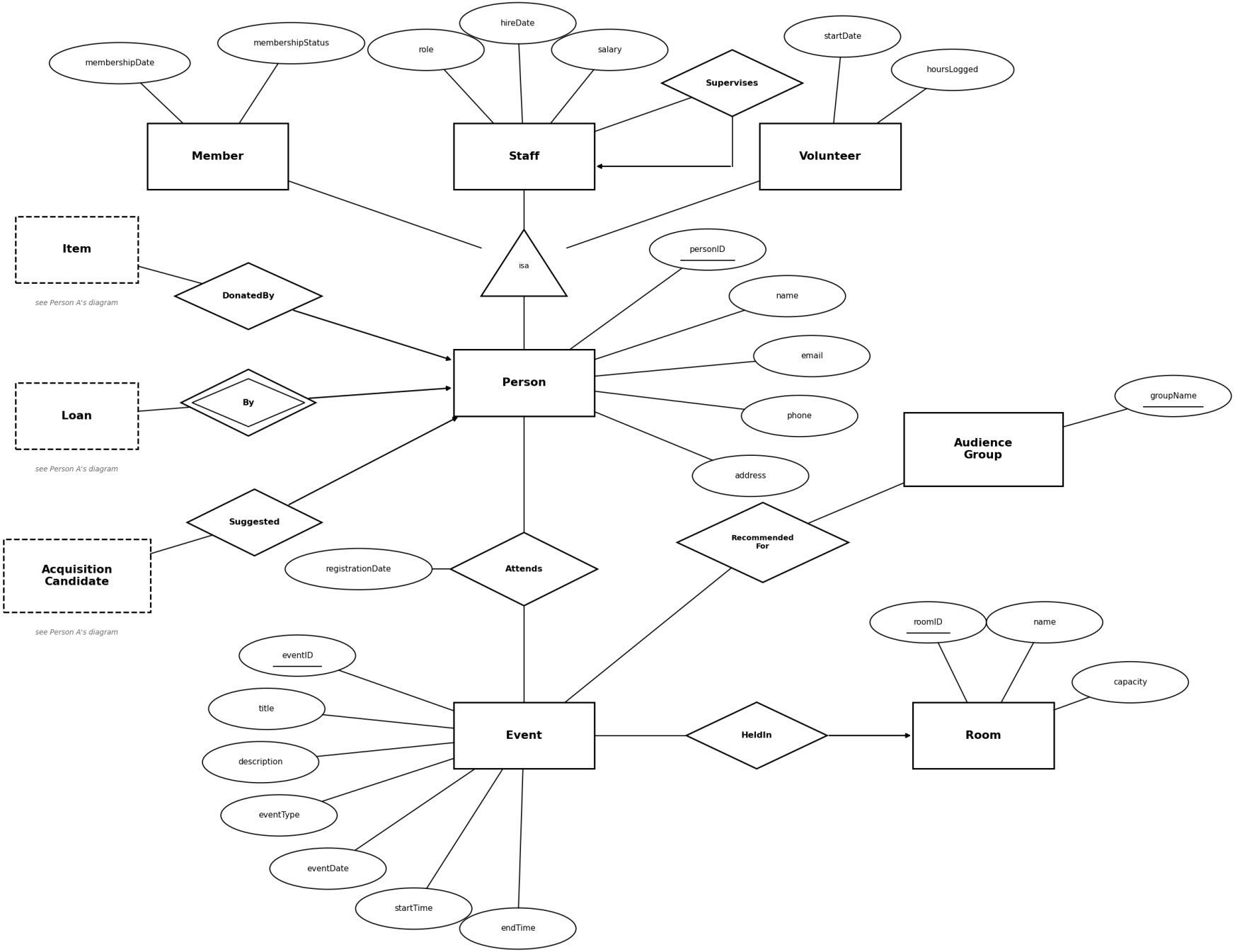
Event Audiences and Registration

Each event may be recommended for any number of audience groups, drawn from a fixed set of five: kids, teens, adults, seniors, and all ages. The same group can of course be recommended many events, and an event with no recommendation is simply open-interest. Separately from recommendations, people register to attend events: any person known to the library, member or not, may register for an event, and the date of registration is recorded. A person can register for a given event at most once, and an event stops accepting registrations once the number of registered attendees reaches the capacity of the room it is held in. Who an event is recommended for and who has actually registered are independent facts, so the library tracks them separately: a recommendation links an event to an audience group, while a registration links an event to an individual person.

Step 2: E/R Diagrams

Notation: rectangle = entity set, double rectangle = weak entity set, oval = attribute (solid underline = key, dashed underline = partial key), diamond = relationship, double diamond = identifying relationship, triangle = isa. Dashed boxes are placeholders marking where a relationship crosses into the other half's diagram (captioned accordingly); the two halves share identical relationship names (DonatedBy, By, Suggested) at every crossing point.


author
ISBN
publisher
PrintBook
shelfLoc.
author
ISBN
fileFormat
OnlineBook
accessURL
pubDate
issueNumber
publisher
Magazine
issueNumber
volume
subjectArea
Journal
format
artist
duration
Recording
isa
itemID
title
acquisitionMethod
dateAdded
status
Item
Person
DonatedBy
see Person B's diagram
For
candidateID
status
proposedTitle
format
dateProposed
estimatedCost
Acquisition Candidate
Suggested
Person
see Person B's diagram
loanDate
returnDate
status
dueDate
Loan
Member
By
see Person B's diagram
Incurs Fine
amountOwed
dateIssued
datePaid
status
Fine


membershipDate
membershipStatus
role
hireDate
salary
startDate
hoursLogged
Supervises
Member
Staff
Volunteer
Item
see Person A's diagram
DonatedBy
isa
personID
name
email
phone
address
Person
Loan
see Person A's diagram
By
Suggested
Acquisition Candidate
see Person A's diagram
registrationDate
Attends
Recommended For
Audience Group
groupName
eventID
title
description
eventType
eventDate
startTime
endTime
Event
HeldIn
roomID
name
capacity
Room

Step 3: Does Your Design Allow Anomalies?

0. Converting the ERD (Step 2) to relations

Each non-weak entity set becomes its own relation. The isa hierarchy uses **Strategy 1 (straight E/R)**: one relation per subclass holding the root key plus its own attributes. Many-one relationships (DonatedBy, Suggested) are folded into the "many" side's relation instead of getting their own relation. Weak entities carry the borrowed key(s) of their supporting entity set(s).

- **Item**(itemID, title, acquisitionMethod, dateAdded, status, donatedBy)
- **PrintBook**(itemID, ISBN, author, publisher, shelfLocation)
- **OnlineBook**(itemID, ISBN, author, fileFormat, accessURL)
- **Magazine**(itemID, issueNumber, publicationDate, publisher)
- **Journal**(itemID, volume, issueNumber, subjectArea)
- **Recording**(itemID, artist, format, duration)
- **Loan**(itemID, personID, loanDate, dueDate, returnDate, status)
- **Fine**(itemID, personID, loanDate, amountOwed, dateIssued, datePaid, status)
- **AcquisitionCandidate**(candidateID, proposedTitle, format, status, dateProposed, estimatedCost, suggestedBy)

(donatedBy and suggestedBy are nullable FKs to Person, reflecting optional participation, so folding them in causes no anomaly.)

---

1. Item(itemID, title, acquisitionMethod, dateAdded, status, donatedBy)

FDs: itemID → title, acquisitionMethod, dateAdded, status, donatedBy No other attribute determines another (title doesn't fix acquisitionMethod, etc.).

Candidate key: {itemID}, since itemID$^+$ = all attributes. **BCNF check:** the only non-trivial FD has itemID (a key) on the left. **Already BCNF.**

---

2. PrintBook(itemID, ISBN, author, publisher, shelfLocation)

FDs:

- itemID → ISBN, author, publisher, shelfLocation (itemID is the key of every subclass relation)
- **ISBN → author, publisher** (real-world assumption: an ISBN identifies one published edition, so the same ISBN always has the same author and publisher, regardless of how many physical copies the library owns)

Candidate key: {itemID}.

BCNF check: is ISBN a superkey? No. The library can hold multiple physical copies of the same book (same ISBN, different itemID, e.g. two copies of the same novel on different shelves), so ISBN → author, publisher **violates BCNF**.

Anomaly this causes if left alone: author/publisher get repeated on every copy row; fixing a

typo in an author's name requires updating every copy of that book instead of one row (update anomaly), and the redundant copies could drift out of sync.

Decomposition:

- **Book**(ISBN, author, publisher), PK = ISBN
- **PrintBook**(itemID, ISBN, shelfLocation), PK = itemID, FK: ISBN → Book

Lossless-join check: common attribute = ISBN, which is a key of Book, so the join is lossless. **Dependency-preserving:** ISBN→author,publisher is enforced in Book; itemID→ISBN, shelfLocation is enforced in the new PrintBook. Both original FDs are covered.

---

3. OnlineBook(itemID, ISBN, author, fileFormat, accessURL)

FDs:

- itemID → ISBN, author, fileFormat, accessURL
- **ISBN → author** (same real-world assumption as above: online copies of the same edition share the same author)

Candidate key: {itemID}. **BCNF check:** ISBN → author violates BCNF for the same reason as PrintBook (the same ISBN can back multiple online copies/licenses, e.g. multiple simultaneous e-book licenses of the same title).

Decomposition: reuse the same Book(ISBN, author, publisher) relation from PrintBook's decomposition (author is the same fact regardless of format, so one shared table avoids storing it twice):

- **OnlineBook**(itemID, ISBN, fileFormat, accessURL), PK = itemID, FK: ISBN → Book

Lossless (ISBN is the key of Book), and dependency-preserving for the same reason as above.

---

4. Magazine(itemID, issueNumber, publicationDate, publisher)

FDs: itemID → issueNumber, publicationDate, publisher.

Unlike ISBN for books, this schema has **no magazine-title identifier** in this relation (issueNumber alone is ambiguous: issue #5 of two different magazines are unrelated), so there is no attribute that determines publisher other than the full key.

Candidate key: {itemID}. **BCNF check:** only FD has a key on the left. **Already BCNF.**

---

5. Journal(itemID, volume, issueNumber, subjectArea)

FDs: itemID → volume, issueNumber, subjectArea. No journal-title identifier exists in this relation either, so no other FD applies.

Candidate key: {itemID}. **Already BCNF.**

---

6. Recording(itemID, artist, format, duration)

FDs: itemID → artist, format, duration. artist does not determine format or duration (the same artist can have recordings in different formats/lengths).

Candidate key: {itemID}. **Already BCNF.**

---

7. Loan(itemID, personID, loanDate, dueDate, returnDate, status)

FDs: {itemID, personID, loanDate} → dueDate, returnDate, status.

Assumption: dueDate is not assumed to be a strict function of loanDate alone (loan periods can differ by item type or membership tier, and staff can grant extensions), so there is no smaller determinant than the full weak key.

Candidate key: {itemID, personID, loanDate} (matches the weak entity's key from the ERD). **BCNF check:** only FD has the full key on the left. **Already BCNF.**

---

8. Fine(itemID, personID, loanDate, amountOwed, dateIssued, datePaid, status)

FDs: {itemID, personID, loanDate} → amountOwed, dateIssued, datePaid, status (key borrowed entirely from Loan, since Fine is 1:1 with Loan and contributes no partial key of its own).

Assumption: amountOwed is not assumed to be a strict function of dateIssued (fine policy/rate can change over time or be manually waived/adjusted), so no smaller determinant exists.

Candidate key: {itemID, personID, loanDate}. **Already BCNF.**

---

9. AcquisitionCandidate(candidateID, proposedTitle, format, status, dateProposed, estimatedCost, suggestedBy)

FDs: candidateID → proposedTitle, format, status, dateProposed, estimatedCost, suggestedBy. proposedTitle does not determine the rest (the same title could be proposed twice, in different formats or at different times, as separate candidates).

Candidate key: {candidateID}. **Already BCNF.**

Final BCNF Schema (Person A's cluster)

```
Item(itemID, title, acquisitionMethod, dateAdded, status, donatedBy)
Book(ISBN, author, publisher)
PrintBook(itemID, ISBN, shelfLocation)
OnlineBook(itemID, ISBN, fileFormat, accessURL)
Magazine(itemID, issueNumber, publicationDate, publisher)
Journal(itemID, volume, issueNumber, subjectArea)
Recording(itemID, artist, format, duration)
Loan(itemID, personID, loanDate, dueDate, returnDate, status)
Fine(itemID, personID, loanDate, amountOwed, dateIssued, datePaid, status)
```

```
AcquisitionCandidate(candidateID, proposedTitle, format, status, dateProposed, estimatedCost,
suggestedBy)
```

Only one real decomposition was needed (Book split out of PrintBook/OnlineBook); every other relation was already in BCNF because its only non-trivial FD already has the full key on the left. No other attribute in this cluster determines another beyond what's explained by the primary key.

0. Converting the ERD (Step 2) to relations

Same conversion rules as Person A's half. Each strong entity set becomes its own relation. The isa hierarchy uses **Strategy 1 (straight E/R)**: one relation per subclass holding the root key plus its own attributes: the right fit here because the hierarchy is overlapping + partial, so a person can appear in several subclass relations or in none. Many-one relationships (HeldIn, Supervises) are folded into the "many" side's relation: HeldIn becomes a NOT NULL roomID in Event (total participation), and Supervises becomes a nullable supervisorID in Staff (the head of the library has none). Many-many relationships (RecommendedFor, Attends) each get their own relation keyed by the pair of participating keys.

- **Person**(personID, name, email, phone, address)
- **Member**(personID, membershipDate, membershipStatus)
- **Staff**(personID, role, hireDate, salary, supervisorID)
- **Volunteer**(personID, startDate, hoursLogged)
- **Room**(roomID, name, capacity)
- **Event**(eventID, title, description, eventType, eventDate, startTime, endTime, roomID)
- **AudienceGroup**(groupName)
- **RecommendedFor**(eventID, groupName)
- **Attends**(personID, eventID, registrationDate)

---

1. Person(personID, name, email, phone, address)

FDs:

- personID → name, email, phone, address
- **email → personID** (real-world assumption from the Step 1 spec: every person has a contact email and no two people share one, since email is how the library reaches a person about loans, fines, and registrations)

Candidate keys: {personID}, and also **{email}**, a second, non-obvious candidate key: from email → personID and personID → everything, transitively email$^+$ = all attributes. The schema enforces it with NOT NULL UNIQUE on email.

BCNF check: both non-trivial FDs have a candidate key on the left (personID and email are each keys). **Already BCNF.**

Alternative assumption: if shared household emails were allowed, email → personID would not hold; email stops being a candidate key (and the UNIQUE constraint would have to go), but no FD with a non-key determinant appears either way, so the relation stays in BCNF under both assumptions. name, phone, and address determine nothing (two people can share any of them).

---

2. Member(personID, membershipDate, membershipStatus)

FDs: personID → membershipDate, membershipStatus. Neither membershipDate nor membershipStatus determines anything (many people join the same day; many memberships share a status).

Candidate key: {personID}, since personID$^+$ = all attributes. **BCNF check:** the only non-trivial FD has a key on the left. **Already BCNF.**

---

3. Staff(personID, role, hireDate, salary, supervisorID)

FDs: personID → role, hireDate, salary, supervisorID.

Assumption: salary is negotiated per person, not fixed by job title (two Librarians hired in different years earn different amounts), so role → salary does **not** hold, and no attribute other than personID determines anything.

Candidate key: {personID}. **BCNF check:** only FD has a key on the left. **Already BCNF.**

Alternative assumption and where it would break: if the library instead paid on a rigid scale where the job title fixes the pay, role → salary would hold with role not a superkey: a **BCNF violation** (salary would be repeated on every staff row with that role: update anomaly). The decomposition would be **RolePay**(role, salary) with PK = role, and **Staff**(personID, role, hireDate, supervisorID) with an FK on role → RolePay. That join is lossless (role is a key of RolePay) and dependency-preserving. We keep the per-person salary design because it matches how the spec describes staff pay.

---

4. Volunteer(personID, startDate, hoursLogged)

FDs: personID → startDate, hoursLogged. Neither startDate nor hoursLogged determines anything.

Candidate key: {personID}. **Already BCNF.**

---

5. Room(roomID, name, capacity)

FDs: roomID → name, capacity.

Assumption: room names are **not** assumed unique (a future branch or renovation could produce two "Meeting Room A"s), so name → roomID is not claimed. If names were assumed unique, {name} would be another candidate key, and the relation would still be in BCNF, since name would then be a key. capacity determines nothing (many rooms can seat 20).

Candidate key: {roomID}. **Already BCNF.**

---

6. Event(eventID, title, description, eventType, eventDate, startTime, endTime, roomID)

FDs:

- eventID → title, description, eventType, eventDate, startTime, endTime, roomID
- **{roomID, eventDate, startTime} → eventID** (real-world rule from the Step 1 spec: a room cannot be double-booked, so at most one event occupies a given room at a given date and start time)

Candidate keys: {eventID}, and also **{roomID, eventDate, startTime}**, the second, non-obvious candidate key: it determines eventID, and eventID determines everything else, so its closure is all attributes. The schema enforces it with UNIQUE(roomID, eventDate, startTime) (and the no-overlap trigger enforces the stronger interval version of the rule).

BCNF check: both non-trivial FDs have a candidate key on the left. **Already BCNF.**

Alternative assumption and where it would break: the analysis does not assume anything like "every BookClub lasts 90 minutes". If eventType → duration were real, endTime would be determined by {eventType, startTime}, a non-superkey determinant: a BCNF violation whose fix would be **TypeDuration**(eventType, duration) with endTime dropped from Event and recomputed on the join. Event lengths genuinely vary here (Step 5 has 60-, 90-, and 360-minute events), so no such FD holds.

---

7. AudienceGroup(groupName)

Single-attribute relation; only trivial FDs exist. **Candidate key:** {groupName}. **Trivially BCNF.**

---

8. RecommendedFor(eventID, groupName)

FDs: none that are non-trivial: the relation is all key. An event can be recommended for many groups and a group can have many events, so neither attribute determines the other.

Candidate key: {eventID, groupName}. **BCNF check:** a relation with no non-trivial FDs cannot violate BCNF. **Already BCNF.**

---

9. Attends(personID, eventID, registrationDate)

FDs: {personID, eventID} → registrationDate (a person registers for a given event at most once, per the Step 1 spec, so the pair fixes the registration date).

Assumption: registrationDate is genuinely determined by the pair, not by either attribute alone (one person registers for many events, and one event has many registrants, on assorted dates).

Candidate key: {personID, eventID}. **BCNF check:** only FD has the full key on the left. **Already BCNF.**

Why RecommendedFor and Attends are two relations, not one

Both hang off Event, so one could imagine a single combined relation R(eventID, groupName,

personID, registrationDate). That design is wrong because the two facts are **independent**: which audience groups an event is aimed at has nothing to do with which individual people have registered. In the combined relation this independence shows up as the multivalued dependencies eventID ↠ groupName and eventID ↠ {personID, registrationDate}, a 4NF violation even though no FD is violated. Concretely, an event recommended for 2 groups with 5 registrants needs 2 × 5 = 10 rows to stay consistent instead of 2 + 5 = 7, every new registrant must be repeated once per group (redundancy + update anomaly), and an event with recommendations but no registrants yet (or vice versa) forces NULLs into half the key. Keeping the two many-many relationships as separate relations, exactly as the ERD draws them, avoids all of this.

Final BCNF Schema (Person B's cluster)

```
Person(personID, name, email, phone, address)
Member(personID, membershipDate, membershipStatus)
Staff(personID, role, hireDate, salary, supervisorID)
Volunteer(personID, startDate, hoursLogged)
Room(roomID, name, capacity)
Event(eventID, title, description, eventType, eventDate, startTime, endTime, roomID)
AudienceGroup(groupName)
RecommendedFor(eventID, groupName)
Attends(personID, eventID, registrationDate)
```

No decomposition was needed: every non-trivial FD in this cluster already has a candidate key on its left side. The two extra candidate keys found along the way, Person.{email} and Event.{roomID, eventDate, startTime}, are declared as UNIQUE constraints in Step 4 so the schema actually enforces what this analysis claims.